Offline Document Workflows on Android: A Source-Level Study of PDF and Office Writers

Ikbal Fadilah
Causyne
ikbal@causyne.com
ORCID: 0009-0008-4853-135X

Abstract

A file extension is a poor test of a document writer. The output may be a valid ZIP archive yet contain broken relationships, or it may open in one reader while losing content. I inspect an Android application's PDF compression and DOCX-to-PDF paths alongside a separate Kotlin prototype that writes minimal DOCX, XLSX, and PPTX packages. The paths have different inputs, dependencies, and maturity. A small fixture test opens each prototype format with Apache POI; the PPTX accepts two requested slides but contains only one. The `copyPdfStream` helper copies bytes. I use these observations to define format-specific checks and an on-device measurement protocol. The report contains no completed cross-reader study, physical-device benchmark, or network trace. It is a bounded implementation study and experimental design, not evidence of four-format production compatibility.

Keywords: Android, offline computing, PDF, Office Open XML, document interoperability, reproducibility

1 Introduction

The starting point for this study was a deceptively simple question: what does it mean for a phone to create a document? Writing bytes proves that an output path ran. It does not show that another application can open the file, that a numeric spreadsheet cell remains numeric, or that a PDF preserves text from the source. An Office file is a package of XML parts joined by relationships; a PDF is a fixed-layout document whose visible pages and extractable text can diverge. Those properties need separate tests.

The practical motivation is local processing of files that users may not wish to upload for conversion. The relevant privacy claim is narrow: an identified operation can be run without transmitting its input. The application under review also contains unrelated network features. A file created on a device is therefore not, by itself, proof of an offline product. Temporary files, imported-document parsers, and logging remain part of the threat model.

I inspected two code boundaries. In the Android application, PDFBox-Android is used for PDF object processing, while Apache POI reads DOCX elements that are redrawn with Android's `PdfDocument`. Separately staged Kotlin code assembles small Office Open XML archives with `ZipOutputStream`. The second codebase has not been demonstrated as an integrated four-format feature of the application. The sections below identify what each path actually writes, where information is lost, and which experiments would establish interoperability. No performance or conformance number is inferred from source inspection.

2 Research questions and claim boundaries

The first question is structural: for each operation, does the output satisfy basic container and part-level requirements? The second is semantic: can an independent reader recover the intended text, numeric values, page order, and slide order? The third is operational: does the operation finish on physical Android hardware without network dependence

or unacceptable memory use? A fourth question concerns loss: what formatting, formulas, images, and accessibility information are discarded or changed during conversion?

These questions require different evidence. ZIP inspection can identify missing or duplicate package parts, but it cannot show that a slide looks correct. A reader opening a document does not establish that every formula or style survived. A successful emulator test does not establish acceptable memory behavior on a low-memory phone. Likewise, a local implementation does not prove zero data egress without observing the tested route while other application traffic is excluded.

The present manuscript distinguishes source observations, the one recorded fixture test, proposed validation, and unsupported claims. In particular, the presence of an Apache POI dependency does not imply full Word compatibility, and the presence of an `.xlsx` filename does not imply that formulas or charts are supported. No DOI, numerical performance result, or independent conformance certificate is asserted.

3 Implementation inventory

In the Android application, `PdfTools.wordToPdf` reads a DOCX through Apache POI, traverses paragraphs and tables, and draws a resulting PDF through Android's `PdfDocument` canvas. This is a content-oriented conversion, not a Word layout engine. The same application has a PDF compression path using PDFBox-Android. Another export route creates chat-derived PDF output through a platform print workflow. These operations have different inputs and fidelity expectations and should not be collapsed into one generic document engine.

The separate Kotlin prototype contains `OfficeDocumentGenerator`, which emits ZIP containers with XML parts for basic word-processing, spreadsheet, and presentation documents. That prototype is a useful starting point for a producer with deliberately narrow output requirements. Its present existence does not demonstrate that all parts conform to ISO/IEC 29500 or that Microsoft Office, LibreOffice, and mobile readers interpret them identically. The prototype's `copyPdfStream` explicitly copies bytes. A byte-for-byte copy can be a legitimate stream operation, but it is not compression and must not be reported as such.

Table 1: Distinct operations found in the inspected source. No row implies independent-reader validation.

Path	Source behavior	Present boundary
Android PDF compression	PDFBox-Android loads the input with temporary-file memory settings and rewrites objects.	Output size and fidelity are not measured here.
Android DOCX to PDF	POI reads paragraphs/tables; Canvas redraws fixed-layout pages.	Word layout and unsupported elements may be lost.
Prototype DOCX/XLSX	ZIP/XML parts are assembled directly; XLSX writes numeric or inline-string cells.	Conformance and reader behavior remain untested.
Prototype PPTX	One slide part is written from `slides.firstOrNull()`.	Additional supplied slides are not serialized.
Prototype PDF stream	`copyPdfStream` copies the input stream.	It is not a compression operation.

The prototype and Android application should be evaluated as separate units. A combined claim would require integration evidence: build configuration, execution on a target Android version, end-to-end file creation, and inspection of the exact bytes produced on device. This manuscript does not infer integration from similar class names or from files staged in the same research workspace. It records what exists and what remains to be shown.

3.1 Android execution and offline scope

Android's `PdfDocument` API [1] provides a way to start a page, draw on its canvas, finish the page, and write the document. It does not supply a full DOCX layout interpreter. In the reviewed conversion path, layout decisions such as wrapping, pagination, font selection, table width, and overflow are made by application code. A correct PDF container can therefore still differ substantially from the source document. The acceptance test should compare rendered pages and extracted content rather than merely checking that `writeTo` returned.

For an offline experiment, airplane mode alone is insufficient evidence. The test should first record whether assets and libraries are already installed, then run the operation with the network disabled and monitor process traffic. Inputs must reside locally and output files must be written to app-private storage or through a user-selected document URI. A test that

opens a document from cloud-backed storage would confound the question. The report should state the Android version, device model, available memory, and whether the run was cold or warm.

The privacy claim must be scoped to the document operation. It should specify what content is read, where temporary files are created, and when they are deleted. The application as a whole may still communicate for unrelated features. If a future implementation logs document text or forwards it to a model service, the local-only claim must be revised. The current document paths have not been accompanied by a captured network trace in this paper, so absence of transmission remains a test target, not a verified result.

3.2 Word-processing output and conversion

DOCX is a package of related XML parts rather than one XML file. A minimal producer must make its content types, package relationships, document part, and document relationships mutually consistent. This can be checked mechanically before opening the file in a reader. The more difficult question is whether the produced document carries the author-intended structure: paragraphs, runs, tables, line breaks, Unicode text, and styles. The staged prototype supports a narrow subset and should be described in those terms.

For DOCX-to-PDF conversion, preservation of content and preservation of appearance must be scored separately. A paragraph may be present in the PDF text layer while its font, indentation, or page break differs. A table may be rendered as text in the right order but with incorrect cell boundaries. A document containing images, floating objects, footnotes, tracked changes, or complex headers is outside the demonstrated scope of the reviewed code. Silence about these features would invite a reader to assume more than the implementation supports.

The proposed corpus should include a small, licensed set of documents with simple paragraphs and tables, plus targeted adversarial cases: non-Latin scripts, right-to-left text, long words, mixed fonts, nested tables, and content extending beyond page bounds. The first group tests the supported subset; the second identifies failure boundaries. Evaluation should capture both a normalized text comparison and a visual comparison of rendered pages. Human review is needed for layout defects that a text diff cannot express.

3.3 Spreadsheet output

An XLSX writer needs more than a ZIP archive containing a worksheet. Workbook and worksheet relationships, content types, namespaces, and references must point to the intended parts. Cells need a type, and a numeric value must remain numeric when opened in a second reader. Text stored as inline strings or shared strings can be valid, but the writer and test must agree on which representation is used. A spreadsheet that renders a number in a cell but stores it as text may fail downstream calculations.

The staged prototype should first be evaluated on plain sheets with strings and numbers. The unit tests should inspect the XML and relationships directly. Integration tests should open each generated file in at least two independent readers, save it, reopen it, and compare cell coordinates and typed values. Formulas require a separate test because a formula string, a cached value, and recalculation behavior are distinct. A file with no formula implementation should say so explicitly, even if the `.xlsx` extension is present.

Useful error cases include empty sheets, large cell indexes, repeated strings, characters requiring XML escaping, negative and decimal numbers, dates, and cells with missing values. The result report should distinguish producer defects from reader differences. A screenshot of one visible worksheet is not enough: it can conceal broken type semantics, missing sheets, or silently repaired package parts. No spreadsheet compatibility percentage is available in the current evidence.

3.4 Presentation output

The prototype's presentation writer accepts a list of slides, but the current method takes `slides.firstOrNull()`, writes one `slide1.xml`, and adds one slide relationship. A call with three supplied slides therefore does not serialize three slides. This is a source-level limitation, not an interoperability measurement. The writer may still be useful for a one-slide text outline, provided a separate reader can open and render that output. It does not currently implement the multi-slide behavior suggested by its input type.

The first acceptance criterion is graph integrity: every slide listed in the presentation has a corresponding part, every referenced layout exists, and XML can be parsed without repair. The second is semantic integrity: expected text appears on the expected slide and in a stable reading order. The third is visual integrity: independent renderers display legible

content without clipping or unexpected overlap. These tests should be repeated after an open-save round trip because some readers silently repair files on first load.

Evaluation cases should include a title slide, multiple slides, long titles, multiline text, punctuation requiring XML escaping, and Unicode. Images, charts, animation, embedded media, speaker notes, and complex themes should be marked unsupported until implemented and tested. A fixture opens the emitted PPTX with Apache POI and counts one slide despite two requested slides. No visual rendering or second reader was tested. That limitation is more informative than an unqualified claim that the file "works."

3.5 PDF operations and loss accounting

PDF creation, compression, and conversion are distinct tasks. Creation asks whether page content can be drawn and written. Compression asks whether the resulting file is smaller while preserving selected visual and semantic properties. Conversion asks whether source content can be represented in a fixed-layout format. A method that merely copies input bytes belongs to none of the compression experiments, even if its name suggests otherwise.

The Android application's PDF compression path should be evaluated with files containing text, vector graphics, photographs, scans, and mixed pages. For each input, record the original and output size, page count, text extraction, and representative page renders. Report both relative file-size change and visible degradation. If the operation increases size or removes selectable text, that outcome belongs in the results. A single size reduction on one PDF is not a general compression result.

The DOCX-to-PDF path needs a separate comparison protocol. For each source document, retain a reference render from an independent application and a render of the Android output. Human reviewers can mark missing text, clipped lines, table errors, pagination changes, and font substitution. Automated image similarity may help locate differences but cannot by itself decide whether a change is acceptable. The PDF must also be checked for extractable text where the input contains text. None of these paired renderings is yet part of the manuscript.

3.6 What the Android PDF paths actually preserve

The compression path first loads a PDF through PDFBox-Android using temporary-file-backed parsing. It rejects an encrypted input before traversing document objects. Image streams above a size threshold can be decoded and re-encoded as JPEG, with dimensions limited according to the selected mode. The method also attempts to deflate other streams and removes document metadata before saving. These are source-level observations. They explain why the operation can be useful for an image-heavy file, but they also show why the word "lossless" needs qualification: even a mode that avoids image re-encoding still passes through the metadata-removal step.

The implementation reports success when a rewritten PDF is saved. It calculates a percentage from input and output sizes, but it does not in this path render the pages, compare extracted text, or verify that every annotation survived. Some failed stream optimizations are caught and skipped. This is a sensible resilience choice for an interactive tool, yet it makes the final behavior document-dependent. An evaluation must record the actual output size even when it grows, and must distinguish an unchanged file from a smaller one. A result message saying a text document is already at minimum size is not proof of a global optimum.

The DOCX-to-PDF path follows a different model. Apache POI reads paragraphs and tables; the application collects runs into text spans, then draws paragraphs and tables on fixed-size Android PDF pages. A paragraph's paint is derived from the first span's font size and whether any span is bold or italic. The text is then joined before layout. That can flatten mixed styling within one paragraph. Table cells are drawn in fixed-height rectangles, and the rendering call truncates each cell's visible text to thirty characters. A long address or formula in a table cell can therefore be lost in the PDF even if POI read it successfully. These are concrete cases for the visual and text-extraction tests.

Both paths operate locally at the method level, but they do not establish an end-to-end "offline document suite." The source does not show a single common document model flowing through PDF, DOCX, XLSX, and PPTX writers in the Android application. Combining them under one performance number would also mix very different work: object rewriting, Word parsing, canvas pagination, and XML package generation. The experiments below keep the operations separate.

3.7 What the prototype package actually serializes

The DOCX writer creates a ZIP with content types, a package relationship, and a word-processing document part. It appends a title and translates a small set of block types into WordprocessingML. It does not load a Word template or

reproduce a source document's layout. The XLSX writer creates workbook and worksheet parts with their relationships. For each table value, it calls `toDoubleOrNull`: values that parse as numbers are written as numeric cells; other values become inline strings. This choice makes ordinary arithmetic data possible, but it can reinterpret a textual identifier such as `0012` as a number and discard its leading zeros.

The PPTX writer advertises a list of slides in its parameter, yet it creates one slide part and one slide relationship. The content comes from `slides.firstOrNull()`, with a default slide if the input list is empty. Its implementation therefore has a one-slide effective capacity at present. That finding is independent of whether the resulting package opens in a presentation viewer. The package also does not demonstrate charts, themes, notes, animations, or an image relationship graph. Those features require more than an output extension and cannot be inferred from the ZIP structure alone.

The prototype's `copyPdfStream` method is deliberately named for its behavior: it reads one file and copies its bytes to another with a fixed buffer. It does not call a PDF parser, alter streams, or optimize images. For this reason it must remain outside any compression comparison with the Android application's PDFBox path. This separation is central to the report. A reader can reproduce the source audit by examining the corresponding methods; a later benchmark must identify which method created each tested artifact.

3.8 Two small inputs that reveal semantic loss

Consider an XLSX request with a column headed `Product ID` and a cell containing the text `0012`. In the prototype writer, each cell string is passed to `toDoubleOrNull`. That string can be parsed as a number, so the worksheet cell is written as numeric data rather than an inline string. The leading zeros are no longer part of the stored value. A spreadsheet viewer may show `12` or `12.0` depending on its formatting rules. This is not a parsing error by the viewer; it follows from the producer's type inference. A useful regression test would open the generated file in an independent reader and check both the displayed text and the underlying cell type.

The same table can contain a numeric quantity `3` that ought to remain numeric. Treating all cells as strings would preserve the identifier but harm arithmetic. The design therefore needs an explicit type in its input model, not a universal rule inferred from the characters. A `Cell.Text("0012")` and a `Cell.Number(3)` distinction would make the writer's intent testable. The present prototype accepts plain strings and chooses at serialization time. This is a source-level observation about the current API, not a claim that a particular reader already rejected its output.

For a presentation fixture, pass two slides: a title slide and a second slide headed `Methods`. The method signature accepts the two-element list, but it constructs `slide1.xml` from `slides.firstOrNull()` and declares one slide in `presentation.xml`. The output has no second-slide part for the reader to display. The simplest test is structural: count the slide relationships and compare the count with the supplied list. A visual check can then confirm what the reader shows. This reveals a mismatch between input type and serialization without requiring any performance benchmark.

A DOCX-to-PDF fixture can expose a different loss. Put a long sentence into one table cell and mix bold and regular text within one paragraph. The Android converter's table drawing calls `text.take(30)` for the cell, so characters after the thirtieth do not reach the canvas. Paragraph layout joins spans into one string and chooses paint properties for the whole paragraph from aggregate run properties. A source file with only short, uniformly styled paragraphs would never reveal these limitations. The test corpus needs intentionally long and mixed-format content, with a reference render and text-extraction comparison.

These examples are deductions from the reviewed code, not measured interoperability results. They also show why a single "document generated" label is inadequate. A file can be structurally readable while changing an identifier, dropping a slide, or truncating a table cell. The evaluation criteria below retain those distinct failures rather than collapsing them into one success bit.

3.9 The actual object and package boundaries

The Android PDF compression path is a PDF object rewrite. It traverses COS streams and treats image subtypes differently from other streams. For sufficiently large decoded images in lossy modes, it recompresses bitmap content as JPEG and updates width, height, bit depth, color space, and filter metadata. For other streams it may apply Deflate when a Flate filter is not already present. The method catches some per-object exceptions and continues; it therefore needs an output-level validity check after saving. A saved file alone does not demonstrate that every stream was preserved correctly.

Metadata removal is part of the same path. The code clears catalog metadata and several information fields before writing the output. This may be desirable for privacy in some contexts, but it is a semantic change to the document. It can also matter to archival users who rely on provenance fields. A validation record should note whether metadata

was expected to remain, whether the input contained forms or annotations, and whether those elements survived. The code comment describes the operation as non-destructive, but a scientific account should report its actual transformations rather than adopt that adjective.

The Office prototype operates at a lower level. Its ZIP writer provides a small fixed set of package parts and relationships; the DOCX body receives text blocks, the XLSX worksheet receives rows and cells, and the PPTX package receives one slide. This bypasses a desktop office process, but it also means the producer is responsible for namespace correctness, escaping, part declarations, and relationship targets. A ZIP library only guarantees an archive container, not an Office-compatible package. ISO/IEC 29500 [3] describes a much larger format family than the subset emitted by these methods.

The relationship graph is a good unit of analysis. For XLSX, the root relationship points to the workbook and the workbook relationship points to a worksheet. For PPTX, the root points to the presentation and the presentation points to the single slide. A test can enumerate package parts, resolve every target relative to its source part, and flag a missing or duplicate target. That test is cheap and deterministic. It still cannot establish visual fidelity. The next stage must open the file in independent applications, record repair warnings, and compare the resulting content with an input manifest.

4 Validation corpus and measures

The study should use inputs that the author has permission to redistribute or, when redistribution is prohibited, should publish generators and descriptive metadata instead of the private files. Each test case needs an identifier, format, feature tags, expected content, expected behavior, and a record of known ambiguity. The corpus should include ordinary examples and stress cases, but the two should be reported separately. Mixing them into one opaque pass rate would make the result hard to interpret.

Structural validity can be expressed as the fraction of outputs passing container, XML, and relationship checks. Reader interoperability can be recorded per application and version: opened without repair, opened after repair, or failed to open. Semantic fidelity should compare paragraphs, table cells, spreadsheet typed values, and slide text in format-specific ways. Visual fidelity requires a stated rendering method and reviewer procedure. Crash rate and time to completion should be measured per operation with a documented timeout.

For a format-specific test set D_f, let S_i, R_i, and M_i indicate that output i passes structural checks, opens in a named independent reader without repair, and preserves the expected manifest. A strict per-file success rate would be

$$P_f = \frac{1}{|D_f|} \sum_{i \in D_f} S_i R_i M_i. \tag{1}$$

The three component rates must also be reported separately. A reader that silently repairs an archive sets $R_i = 0$ under this definition. No value for P_f has been measured for this report.

The experimental unit matters. Repeated runs of one file are not independent documents. Report both per-file outcomes and repeated-run variability. If a suite contains ten simple documents and one hundred near-duplicates, an aggregate percentage will overstate generality. A useful report includes counts per feature class, denominators for every metric, and confidence intervals where sampling justifies them. At present, the benchmark has not been assembled, so this manuscript contains no numerical table of results.

4.1 A staged conformance and interoperability procedure

Testing should move from cheap, deterministic checks to expensive human review. The first stage examines the bytes: confirm the advertised file type, reject truncated ZIP archives, list package members, and verify that no package entry escapes the extraction root. For Office Open XML, parse every XML part and resolve each internal relationship. A broken relationship should be a test failure even if one forgiving viewer manages to display the file. For PDF, record whether a parser can read the cross-reference structure and count pages. A parser success is only the beginning of validation.

The second stage checks expected content against a manifest written before generation. For a DOCX, that manifest can enumerate paragraphs and table cells. For a spreadsheet, it can enumerate sheets, cell addresses, types, and values. For a presentation, it can enumerate slide order and text runs. For a PDF, it can specify page count and text expected on each page. The manifest should preserve deliberate omissions: if a conversion is documented as losing floating images or comments, those are limitations to be reported, not silently ignored. A generated file must not be judged only by the same code that produced it.

The third stage opens each file in independent applications and records any repair prompt. This stage should preserve the original file hash, because some readers rewrite a document as they open or save it. After an open-save round trip, compare the new file against the manifest and inspect the rendered output. The procedure should record the reader name and version, operating system, and exact outcome. ‘Opened" and ‘opened after repair” are different categories. For an offline claim, the test application should be disconnected while the producer runs; independent desktop readers may be used later for interoperability analysis without changing the production claim.

The final stage is human review of representative pages, slides, and tables. Two reviewers should independently mark whether text is missing, clipped, illegible, reordered, or semantically altered. Disagreements can be adjudicated with the source file and manifest visible. This process creates an auditable record rather than a single subjective approval. The resulting paper should publish examples of both successful and failed cases, with identifiers that link back to redistributable test inputs. Only the narrow Apache POI fixture test has been completed; the full procedure has not.

4.2 A format-specific fixture matrix

The protocol becomes reproducible only when each format has a small, explicit test matrix. A DOCX producer fixture can contain two paragraphs, one table, an ampersand, an apostrophe, a non-ASCII character, and a line break. The input manifest should state the exact Unicode strings and paragraph order. Its structural check should resolve the package-level office-document relationship and parse the document part. Its semantic check should extract paragraphs and table cells with an independent library. A visual check can then inspect whether the text is legible in more than one reader. These three checks answer different questions; success in one should not be substituted for another.

The Android DOCX-to-PDF path needs a separate input matrix because it consumes an existing Word file. One fixture should mix bold and ordinary runs in a paragraph. Another should put a sentence longer than thirty characters in a table cell, with distinctive final words so truncation is obvious. A third should include enough paragraphs to cross a page boundary. For each fixture, save a reference render, the Android-produced PDF, extracted text from both, and raster images of corresponding pages. The source trace predicts mixed-style flattening and table-cell truncation; it does not establish the exact appearance of a given reader’s rendering. Those visible outcomes must be recorded from the files produced in a test run.

For XLSX, the smallest useful matrix is not merely an empty workbook and one string. It should include `0012` as a textual product identifier, `3` as a numeric quantity, a negative value, a decimal value, an empty cell between populated cells, and a string containing XML-reserved characters. The expected manifest needs an explicit type for each cell. If the producer interface accepts only strings, the test should record that limitation and the type chosen by `toDoubleOrNull`. A reader test should inspect both the displayed text and the stored cell type after reopening the file. A screenshot showing `12` is insufficient to determine whether the original identifier survived.

Spreadsheet round trips introduce another distinction. A reader may preserve the input value but change its representation when saving. The test should compare the producer’s original package, the reader’s loaded object model, and the reader’s saved package separately. If a numeric cell becomes `12.0` in XML after an open-save round trip, that may be a harmless serialization change for a quantity but a semantic loss for an identifier. The manifest should express the intended domain meaning before the writer runs, so the interpretation is not chosen after seeing the output.

For PPTX, create one-slide, two-slide, and empty-list requests. The current method’s use of `slides.firstOrNull()` predicts that the two-slide request yields only one serialized slide and that the empty request uses its default slide. A structural test can count slide parts, presentation slide IDs, and slide relationships. A semantic test can search for a distinct phrase placed only on the second requested slide. A reader test can report whether the single emitted slide opens without repair and whether it clips a long title. These results should be kept separate: a package may open correctly while failing to represent the requested slide list.

PDF fixtures must distinguish byte copying, object rewriting, and canvas drawing. `copyPdfStream` can be tested with a known input hash; equality of input and output bytes is the expected behavior and cannot count as compression. For PDFBox rewriting, use at least a text-only page, a JPEG-heavy page, and a mixed page with metadata. Record input and output hashes, byte sizes, page counts, extractable text, metadata, and page renders. In lossy mode, inspect image degradation; in a nominally lossless mode, still check metadata because the source clears it. For the canvas converter, compare content and layout against the DOCX source rather than against a PDF compression baseline.

Every fixture should have a failure record with the same fields: stage, observed bytes or reader behavior, expected manifest item, first divergence, and whether the output remains usable for the intended task. A missing slide and a reader repair warning should not share a generic “failed” label without explanation. A failed case can reveal a precise engineering change: add a slide relationship per input slide, preserve a typed spreadsheet identifier, or stop truncating cell text. The paper should report the tested source revision for each case so that a later fix is not confused with an earlier

observation.

The matrix should also specify limits before execution. A minimal DOCX writer need not be judged against tracked changes or embedded media if it never claims them. Conversely, a method accepting a list of slides must be tested on a list of at least two elements. The declared input contract determines the minimum fair test. Cases outside the declared contract can still be included as exploratory stress tests, but they belong in a separate denominator. This prevents a narrow producer from being condemned for unclaimed features and prevents a broad API signature from hiding unimplemented behavior.

Finally, retain the actual generated files and a compact machine-readable result for each fixture. Source inspection identifies where to look; only these artifacts can show what independent applications did with the bytes. The matrix above is a proposed experiment. The one Apache POI fixture confirms parser acceptance of three simple prototype outputs, not cross-reader interoperability or PDF behavior.

Table 2: Minimal proposed document fixtures and their distinct acceptance tests.

Operation	Input that exposes a boundary	Required observation
DOCX to PDF	Mixed-style run; table cell over 30 characters.	Extracted text and page render against source.
XLSX writer	Text ID `0012`; numeric quantity `3`.	Stored cell types and independent-reader values.
PPTX writer	Two slides with unique second-slide text.	Slide parts, relationships, and displayed order.
PDF stream copy	Known file and hash.	Byte equality, with no compression claim.
PDF object rewrite	Text page, image page, and metadata.	Size, pages, text, metadata, and rendered pixels.

4.3 Evaluation design and analysis rules

The primary outcome for a producer should be a per-file success indicator with three conditions: structural checks pass, at least one independent reader opens without repair, and expected content is recoverable. This compound outcome is intentionally strict. A secondary report should show each component separately, because a low compound score does not explain whether the failure was a missing ZIP relationship, a semantic mismatch, or a rendering defect. For conversions, a separate loss inventory is more informative than a binary pass mark.

The corpus must be split into design-time and held-out sets by template family or source. Otherwise a writer may be tailored to a handful of examples and appear to generalize only because the test set repeats the same structure. If baselines are used, they should receive the same input documents and comparable output settings. A desktop office suite is a useful compatibility reference but not a fair mobile resource baseline. Resource comparisons should be limited to implementations run under similar conditions, or the asymmetry should be made explicit.

Performance analysis should report median, interquartile range, and a tail statistic with the number of measured files and runs. Failures and timeouts should remain in the denominator of success rates. When a document is too large for the implementation, the paper should report the size and failure mode rather than omit it. For visual evaluation, identify the rendering application and page rasterization settings; otherwise the same file may appear different because of font substitution. For privacy evaluation, identify exactly which operation and process were monitored and what constitutes an attempted connection.

Interpretation must remain proportionate to evidence. A small corpus can support a demonstration and failure analysis, not a sweeping interoperability claim. A measured zero network requests in one test does not prove that every code path is offline. A few examples opening in one reader do not prove ISO conformance. These cautions are not barriers to publication; they are the boundary conditions that let a reader judge what was actually shown. The present draft contains protocol rather than outcomes, so these metrics are specifications for future measurements.

4.4 Device protocol and reproducibility

Physical-device tests should specify Android release, chipset, RAM, free storage, application build identifier, and library versions. Cold and warm runs should be distinguished. Before each cold run, clear only the relevant process state while retaining the same installed components; do not silently change downloaded assets between conditions. Time the operation from user-visible invocation to durable output, and record file size and page or sheet count. Peak Java heap

alone is insufficient because PDF processing and ZIP operations can allocate native buffers or temporary files.

Memory observations should include Java heap, native memory if available, and process RSS or an equivalent system measure. Storage observations should include temporary-file peaks as well as final output size. For large documents, note whether the implementation streams or holds entire content in memory. Capture crashes, out-of-memory failures, timeouts, and malformed outputs as failures rather than discarding them from latency summaries. If throttling is likely, randomize run order and report device temperature or a consistent rest interval.

Reproducibility requires exact test inputs or generators, a buildable source revision, a command to run validation, and machine-readable results. The release should identify the version studied and include the fixture manifests. The present manuscript is a draft until these artifacts and measurements are complete.

5 Threats to validity and security

Document formats are large and permissive. A producer may pass self-tests because the same code writes and reads its own assumptions. Independent readers reduce that bias, but those readers can repair malformed packages silently. Save-and-reopen checks and inspection of repair warnings are necessary. Cross-platform rendering differences can also make pixel-level comparison misleading; the evaluation should combine text, structure, and human visual review rather than use one metric alone.

Imported documents can be hostile. ZIP bombs, duplicate part names, oversized images, malformed XML, and path traversal attempts should be handled without exhausting device resources or writing outside the selected destination. Apache POI's security guidance [2] warns that untrusted documents can consume substantial time and memory and recommends additional containment. This report has not audited the application's exact dependency tree or proven that every parsing route is safe. Such claims require separate review and version pinning.

The privacy threat model includes local temporary files and Android document-provider behavior, not just network requests. Deleting a temporary file after use may still leave copies in backups or external providers. A future study should document storage locations, retention, logs, crash reports, and permissions for the tested build. These risks do not negate the value of local processing, but they make the boundary precise. No absolute security or privacy guarantee is claimed here.

5.1 Data stewardship and release boundaries

A reproducible paper does not require publishing private user documents. Synthetic examples can cover many structural features, and public-domain or author-created documents can cover visual cases. The test manifest should include provenance, redistribution permission, and a short description of the feature being tested. Files supplied by users or captured from private correspondence should stay out of the repository and the submission archive. A corpus with uncertain rights should be replaced or reduced before release, not described as public by assumption.

The code release should separate original implementation from third-party libraries. An MIT license on author-written extraction code does not relicense Android, Apache POI, PDFBox, or any embedded model. The README should state the boundaries of the code package, supported formats and features, build instructions, and known failures. A release tag should identify the exact revision used in the paper. If a paper and a software archive receive separate persistent identifiers, the metadata should link them as related objects rather than claim that one DOI identifies both artifacts.

Automated scanning should search the release candidate for API keys, tokens, private service URLs, account identifiers, local user paths, and copied personal documents. That scan is a gate, not a proof of safety: reviewers should inspect the actual source tree and generated archive. A build script that reads from an unrestricted workspace can accidentally include files that were never intended for publication. An explicit allowlist of source, tests, documentation, and license files makes the release surface easier to audit.

The paper itself should not present a personal anecdote as evaluation evidence. If privacy or offline operation is a research claim, record the relevant method and observation. If interoperability is a claim, publish the reader versions and failure cases. If a test cannot be executed, keep the language prospective. This discipline protects the author as much as the reader: later iterations can add genuine measurements without needing to retract invented ones.

6 Discussion and next steps

The most defensible near-term result is a narrow implementation report: a local PDF and DOCX workflow in the Android application, and a separately staged package that can generate elementary Office-format files. The immediate work is

to execute the validation protocol. The author should first make a small public test corpus with clear expected outputs, then run container checks, independent-reader tests, physical-device tests, and offline network observation. Failed cases should remain visible in the release record.

Only after those tests should the manuscript be revised into an empirical paper. The revised version would replace planned methods with performed methods, identify exact source revisions, provide results with denominators and uncertainty, compare meaningful baselines, and discuss observed failures. If the prototype is later integrated into the Android product, that integration should be described as a new version with its own tests. The current draft intentionally does not claim that this future work has happened.

The experiments should be run before this report is revised into a compatibility claim. The present source audit is useful because it identifies tests that can falsify overly broad claims at low cost, especially the two-slide and typed-cell cases. A later version can report the observed outcomes without changing the meaning of the methods defined here.

References

[1] Android Developers. *PdfDocument API reference*. `https://developer.android.com/reference/android/graphics/pdf/PdfDocument`

[2] Apache Software Foundation. *Apache POI Security Guidance*. `https://poi.apache.org/security.html`

[3] ISO/IEC. *ISO/IEC 29500-1:2016, Office Open XML File Formats, Part 1*. `https://www.iso.org/standard/71691.html`

Copyright 2026 Ikbal Fadilah. This manuscript is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).